

25.K 个一组翻转链表

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Hard (69.22%)	2553	0	-	-	0

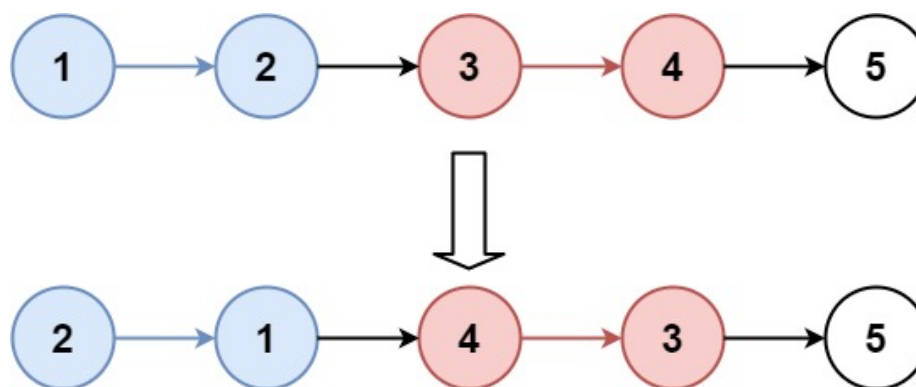
- Tags
- Companies

给你链表的头节点 `head`，每 k 个节点一组进行翻转，请你返回修改后的链表。

k 是一个正整数，它的值小于或等于链表的长度。如果节点总数不是 k 的整数倍，那么请将最后剩余的节点保持原有顺序。

你不能只是单纯的改变节点内部的值，而是需要实际进行节点交换。

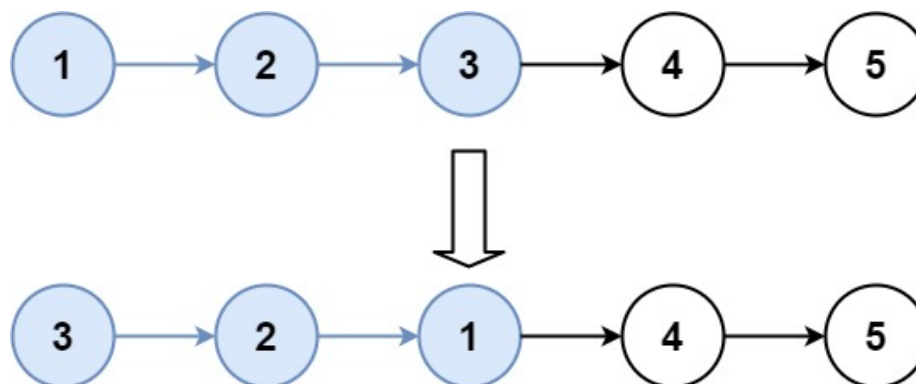
示例 1:



输入: `head = [1,2,3,4,5]`, $k = 2$

输出: `[2,1,4,3,5]`

示例 2:



输入: `head = [1,2,3,4,5]`, `k = 3`

输出: `[3,2,1,4,5]`

提示:

- 链表中的节点数目为 `n`
- $1 \leq k \leq n \leq 5000$
- $0 \leq \text{Node.val} \leq 1000$

进阶: 你可以设计一个只用 $O(1)$ 额外内存空间的算法解决此问题吗?

Discussion | Solution

方法

LeetCode 25: K 个一组翻转链表 (详细解析)

1. 问题理解

目标: 给定一个单链表, 每 `k` 个节点一组进行翻转, 最后不足 `k` 个的节点保持原有顺序。

输入:

- 单链表的头节点 `head`
- 一个正整数 `k` (分组的大小)

输出: `K` 个一组翻转后的链表的头节点。

示例:

- 输入: `head = [1,2,3,4,5]`, `k = 2`
- 输出: `[2,1,4,3,5]` (1,2 翻转 -> 2,1; 3,4 翻转 -> 4,3; 5 不足一组, 保持不变)
- 输入: `head = [1,2,3,4,5]`, `k = 3`
- 输出: `[3,2,1,4,5]` (1,2,3 翻转 -> 3,2,1; 4,5 不足一组, 保持不变)

关键点:

- 必须是实际的节点翻转, 不能只修改节点值。
- `k` 是一个正整数, 且小于或等于链表长度。
- 如果最后剩余的节点不足 `k` 个, 则保持原样。

2. 普通人的思路 (及遇到的困难)

- 与简单翻转的区别: 这不是翻转整个链表, 而是分段翻转。
- 分组: 如何准确地确定每一组的开始和结束节点?
- 连接: 翻转一组后, 如何将翻转后的组与前一部分和后一部分正确地连接起来?
- 不足 `k` 个的处理: 如何判断最后一组是否足够 `k` 个, 并正确处理?

- 指针管理：在翻转和连接过程中，需要非常小心地管理多个指针，防止链表断裂或丢失节点。

3. 核心方法：迭代分组翻转

核心思想：

1. 遍历链表，以 k 个节点为单位进行分组。
2. 对于每一组，判断其长度是否达到 k 。
3. 如果达到 k ，则翻转这一组内的 k 个节点。
4. 将翻转后的子链表重新连接回主链表。
5. 如果不足 k 个，则保持该组节点不变，并结束处理。

辅助工具：

- 哑节点 (**Dummy Node**)：同样用于简化头节点的处理，特别是第一组的翻转连接。
- 指针：
 - `prev`：指向当前处理组的前一个节点。用于连接翻转后的组。
 - `end`：指向当前处理组的最后一个节点。用于判断是否足够 k 个节点，并作为翻转后的新头节点。
 - `start`：指向当前处理组的第一个节点。翻转后会成为该组的新尾节点。
 - `nextGroupStart`：临时存储下一组的起始节点。

详细步骤

1. 创建哑节点 `dummy`, `dummy.next = head`。
2. 初始化 `prev = dummy`。
3. 循环开始：
 - a. 确定分组范围：
 - b. `start = prev.next` (当前组的起始节点)。
 - ii. `end = prev`。从 `prev` 开始，向后移动 k 次，尝试找到 `end`。
 - iii. 在移动过程中，如果 `end` 变为 `null`，说明剩余节点不足 k 个，直接 `break` 循环。
 - iv. 如果成功移动 k 次，`end` 指向当前组的最后一个节点。
 - b. 准备翻转：
 - c. `nextGroupStart = end.next` (保存下一组的头节点)。
 - ii. 断开连接： `end.next = null` (将当前组与链表剩余部分断开，以便独立翻转)。
 - iii. 翻转当前组：
 - iv. 调用一个辅助函数（或内联实现）翻转从 `start` 到 `end` 的子链表。
 - v. 翻转后，原来的 `start` 变成了新尾，原来的 `end` 变成了新头。
 - vi. 重新连接：
 - vii. `prev.next = end` (将前一部分连接到翻转后组的新头节点 `end`)。
 - viii. `start.next = nextGroupStart` (将翻转后组的新尾节点 `start` 连接到下一组的头节点)。

- e. 更新 `prev`:
 - f. `prev = start` (将 `prev` 移动到当前处理完的组的末尾，即原来的 `start` 节点，为下一轮循环做准备)。
4. 循环结束：返回 `dummy.next`。

辅助函数：翻转链表 (Reverse Sublist)

这个函数接收一个链表的头节点，将其翻转，并返回翻转后的新头节点。可以使用标准的迭代法实现（如 LeetCode 206）。

```
private ListNode reverse(ListNode head) {
    ListNode prev = null;
    ListNode curr = head;
    while (curr != null) {
        ListNode next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    return prev; // prev is the new head
}
```

4. 代码实现

5.reverse-nodes-in-k-group.java

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    /**
     * K 个一组翻转链表
     * @param head 链表头节点
     * @param k 每组的节点数
     * @return 翻转后的链表头节点
     */
    public ListNode reverseKGroup(ListNode head, int k) {
        // 哑节点
        ListNode dummy = new ListNode(0);
        dummy.next = head;
```

```

// prev 指向待翻转子链表的前一个节点
ListNode prev = dummy;
// end 指向待翻转子链表的最后一个节点
ListNode end = dummy;

while (true) {
    // 1. 找到 k 个节点，确定 end 的位置
    for (int i = 0; i < k && end != null; i++) {
        end = end.next;
    }
    // 如果 end 为 null，说明剩余节点不足 k 个，结束循环
    if (end == null) {
        break;
    }

    // 2. 准备工作
    ListNode start = prev.next;          // 待翻转子链表的头节点
    ListNode nextGroupStart = end.next; // 下一组的头节点

    // 3. 断开连接，独立出待翻转的子链表
    end.next = null;

    // 4. 翻转子链表，并重新连接
    prev.next = reverse(start); // prev 连接到翻转后的新头（原来的 end）
                                // 注意：reverse 函数返回的是翻转后的头节点

    // 5. 将翻转后的子链表尾部连接到下一组的头节点
    // 翻转后，原来的 start 节点变成了尾节点
    start.next = nextGroupStart;

    // 6. 更新 prev 和 end，为下一轮做准备
    prev = start; // prev 移动到当前已处理部分的末尾
    end = prev;   // end 重置，准备寻找下一组的末尾
}

return dummy.next;
}

/**
 * 翻转一个单链表（迭代法）
 * @param head 要翻转的链表的头节点
 * @return 翻转后的链表的头节点
 */
private ListNode reverse(ListNode head) {
    ListNode prev = null;

```

```

        ListNode curr = head;
        while (curr != null) {
            ListNode nextTemp = curr.next;
            curr.next = prev;
            prev = curr;
            curr = nextTemp;
        }
        return prev; // prev 最终指向原链表的尾节点，即新头节点
    }
}

```

5. 代码解析

- 哑节点 **dummy**: 作用同前，简化边界处理。
- **prev** 和 **end** 指针: **prev** 标记每一组开始翻转前的位置，**end** 用于探测并标记每一组的结束位置。
- 外层 **while(true)** 循环: 不断尝试寻找并处理 **k** 个节点的组。
- 内层 **for** 循环: 用于将 **end** 指针向前移动 **k** 步，同时检查是否提前到达链表末尾。
- 判断 **end == null**: 这是处理最后不足 **k** 个节点的关键。如果 **end** 为 **null**，说明凑不够 **k** 个，直接 **break**。
- **start** 和 **nextGroupStart**: 临时存储当前组的开始节点和下一组的开始节点，用于后续的连接。
- **end.next = null**:: 断开当前组与后续链表的连接，形成一个独立的子链表，方便调用 **reverse** 函数。
- **prev.next = reverse(start)**:: 调用 **reverse** 函数翻转子链表。**reverse(start)** 返回的是翻转后子链表的新头节点（即原来的 **end** 节点），将其连接到 **prev** 后面。
- **start.next = nextGroupStart**:: 将翻转后子链表的新尾节点（即原来的 **start** 节点）连接到下一组的头节点。
- 更新 **prev = start**; **end = prev**:: 将 **prev** 移动到当前处理完的组的末尾（现在是 **start** 节点），并将 **end** 重置到 **prev** 的位置，准备下一轮的探测。
- **reverse** 辅助函数: 标准的链表翻转实现。

6. 复杂度分析

- 时间复杂度: $O(N)$ 。虽然看起来有嵌套循环，但每个节点最多被访问两次（一次是 **end** 指针探测，一次是 **reverse** 函数内部翻转），所以总体是线性的。
- 空间复杂度: $O(1)$ 。只使用了常数个额外的指针变量。辅助函数 **reverse** 也是 $O(1)$ 空间。

7. 如何在面试中讲解

1. 理解问题和目标:

- “目标是将链表每 k 个节点一组进行翻转，不足 k 个的保持不变。需要实际修改节点指针。”
2. 核心思路概述：
 - “我们可以使用迭代的方法，一次处理链表中的一个 k 节点组。”
 - “关键在于：如何准确找到每组的起止点，如何翻转这一组，以及如何将翻转后的组正确连接回主链表。”
 3. 引入关键指针和哑节点：
 - “为了方便操作，我们使用一个哑节点 `dummy`。”
 - “我们需要一个 `prev` 指针，指向当前处理组的前一个节点。”
 - “还需要一个 `end` 指针来探测当前组的末尾。”
 4. 详细描述迭代步骤：
 - “我们循环处理链表。在每次循环开始时，`prev` 指向上一组的末尾（或 `dummy`）。”
 - “首先，我们尝试从 `prev` 向后移动 k 步来找到 `end` 节点。如果在移动过程中 `end` 变为 `null`，说明剩余节点不足 k 个，我们停止处理。”
 - “如果找到了 `end`，我们记下当前组的起始节点 `start = prev.next` 和下一组的起始节点 `nextGroupStart = end.next`。”
 - “然后，我们断开当前组与后面的连接 (`end.next = null`)。”
 - “接着，我们调用一个翻转链表的辅助函数（或者直接实现翻转逻辑）来翻转从 `start` 到 `end` 的子链表。翻转后，`end` 变成了新头，`start` 变成了新尾。”
 - “下一步是重新连接：将 `prev.next` 指向翻转后的新头（即 `end`），并将翻转后的新尾（即 `start`）的 `next` 指向 `nextGroupStart`。”
 - “最后，更新 `prev` 指针，将其移动到当前处理完的组的末尾（即 `start` 节点），为下一轮循环做准备。”
 5. 解释辅助函数（如果使用）：
 - “翻转子链表的部分可以封装成一个独立的 `reverse` 函数，它接收一个头节点，返回翻转后的新头节点，使用标准的迭代法实现即可。”
 6. 处理边界和结束条件：
 - “哑节点处理了第一组的连接问题。”
 - “通过检查 `end` 是否为 `null` 来判断剩余节点是否足够 k 个，从而正确处理链表末尾不足 k 个节点的情况。”
 7. 分析复杂度：
 - “时间复杂度是 $O(N)$ ，因为每个节点最多被访问常数次。”
 - “空间复杂度是 $O(1)$ ，只使用了固定的额外指针。”

这种迭代方法逻辑清晰，空间效率高，是解决此类问题的常用且推荐的方案。

找到具有 1 个许可证类型的类似代码